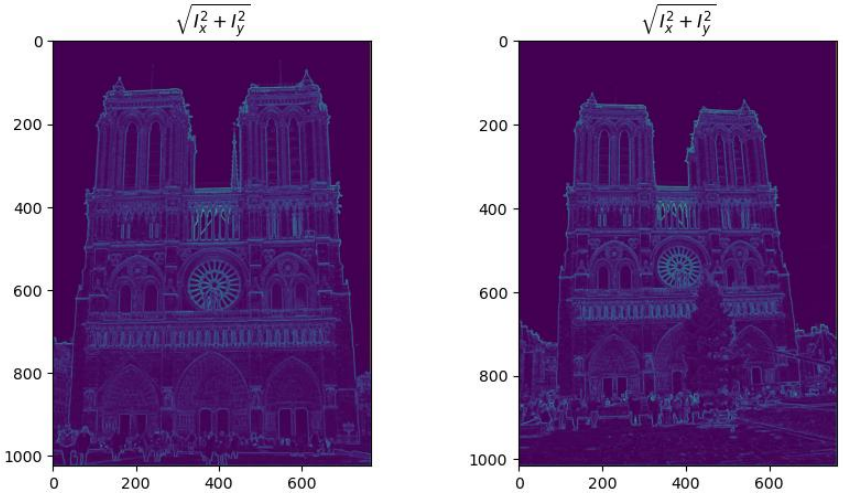
Part 1: Harris corner detector

Insert visualization of \sqrt($I_x^2 + I_y^2$) for Notre Dame image pair from proj2.ipynb here

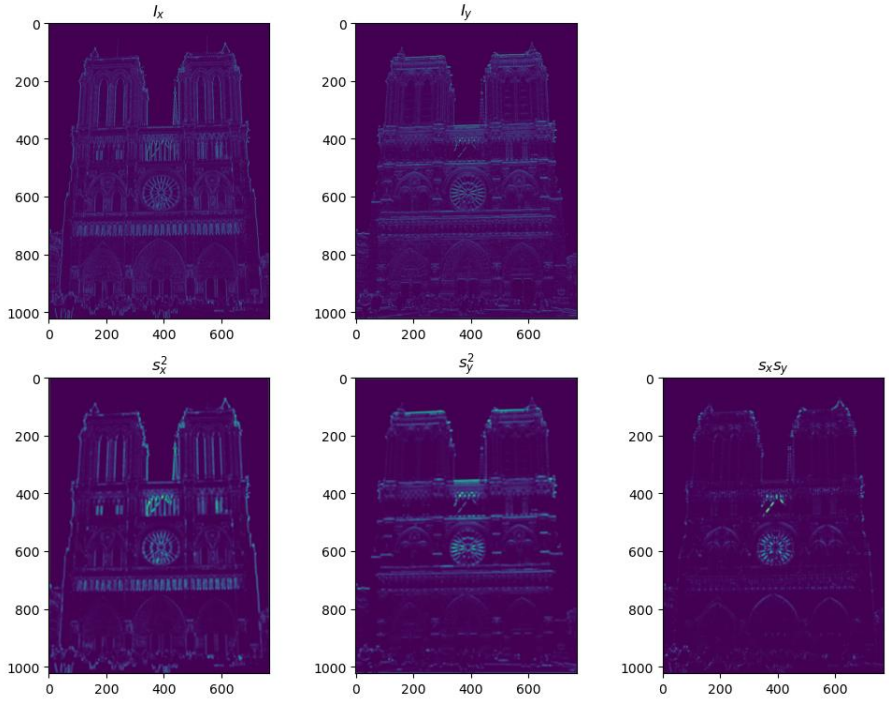


Which areas have highest magnitude? Why?
Edges and high contrast boundaries - windows, strong texture boundaries - have the highest magnitude because intensity changes the most rapidly there.

This is the gradient magnitude, which shows how strongly pixel intensity changes locally. Harris uses gradients to form the second moment matrix and distinguish corners vs edges vs flat regions.

Part 1: Harris corner detector

Visualization of I_x, I_y, s_x^2, s_y^2, $s_x s_y$ for Notre Dame image



Briefly explain what these values represent and why we need them for the Harris corner detector.

Ix and Iy are the image partial derivatives that show the horizontal/vertical intensity change. Sx2 and Sy2 is G * Ix2/Iy2, and this smooths the gradient energy in x/y. SxSy is G * (IxIy), which is the smoothed correlation between x and y gradients.

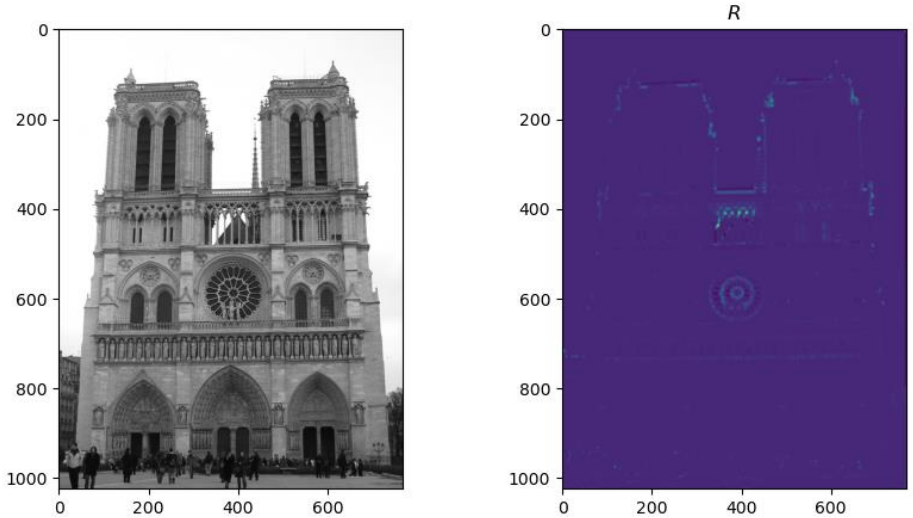
Part 1: Harris corner detector

Why do we convolve the image with a Gaussian filter?

This is done to get gradient information over a neighborhood and reduce noise sensitivity. The Gaussian acts like a weighted local average, so the second-moment estimates are stable and represent the local patch instead of simple noisy pixels.

Part 1: Harris corner detector

Insert visualization of corner response map of Notre Dame image from proj2.ipynb here

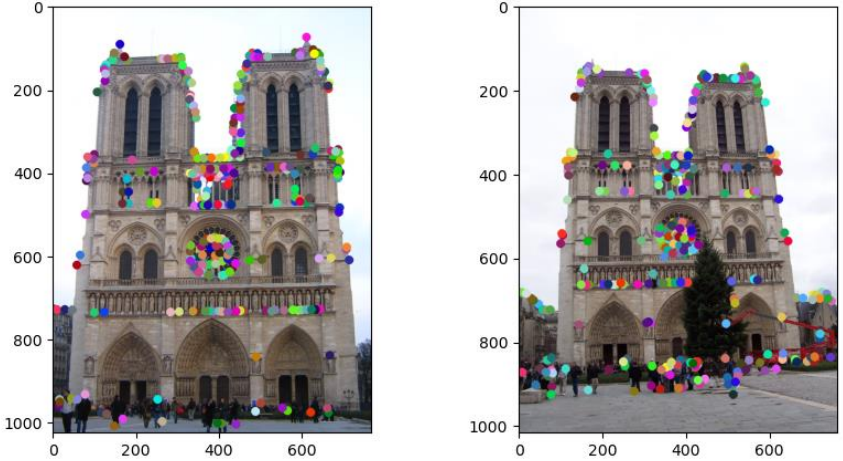


Are gradient features invariant to both additive shifts (brightness) and multiplicative gain (contrast)? Why or why not? See Szeliski Figure 3.2

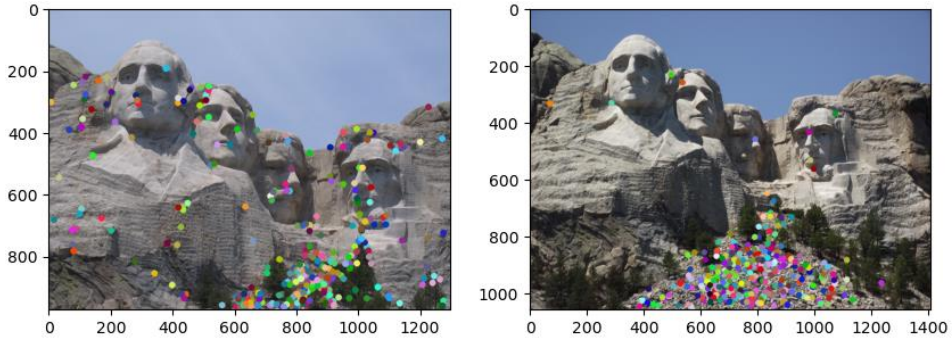
It is invariant to additive brightness shift, since gradients don't change because derivatives remove constants (ex: I' = I + b)
It is not invariant to multiplicative contrast gain, since gradients are scaled (ex: I' = aI). Unless normalized later, the magnitude changes.

Part 1: Harris corner detector

insert visualization of Notre Dame interest points from proj2.ipynb here

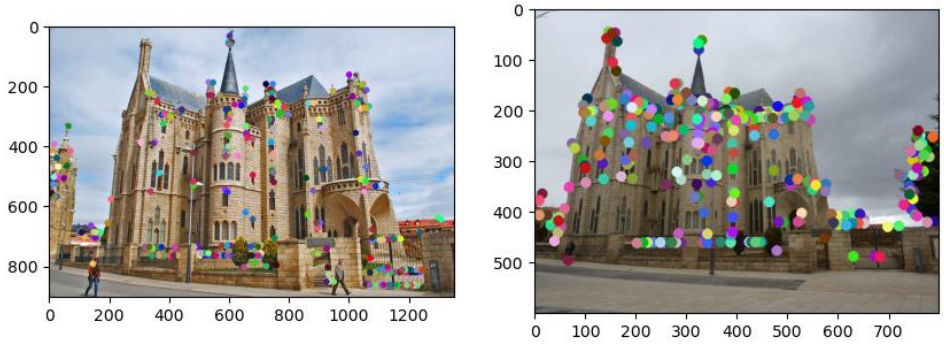


Insert visualization of Mt. Rushmore interest points from proj2.ipynb here



Part 1: Harris corner detector

Insert visualization of Gaudi interest points from proj2.ipynb here



What are the advantages and disadvantages of using maxpooling for non-maximum suppressionn (NMS)?

Advantages are that its quick, simple, GPU friendly, and avoids per pixel comparisons manually.
Disadvantages are that it can keep too many points in a textured area, can suppress nearby strong corners if the window is large, and is sensitive to the chosen pooling window size.

Part 1: Harris corner detector

Explain the intuition behind the Harris corner score equation $R = \det(A) - \alpha \cdot \text{trace}(A)^2$

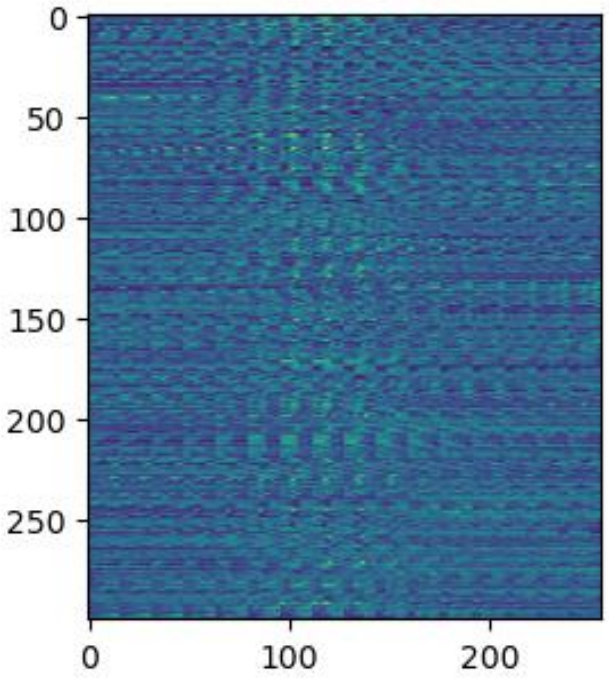
Harris measures the intensity change in both directions. If the structure has two large eigenvalues, moving the window in any direction changes the appearance, meaning it's a corner. The equation is high when both eigenvalues are high.

What is your intuition behind what makes the Harris corner detector effective?

It's a strong local way to detect points that are repeatable under a small illumination changes, since corners are stable, distinctive, and easier to match than edges or flat areas.

Part 2: Normalized patch feature descriptor

Insert visualization of normalized patch descriptor from proj2.ipynb here

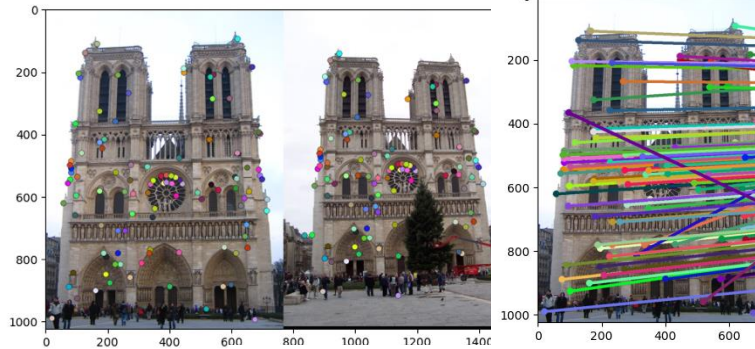


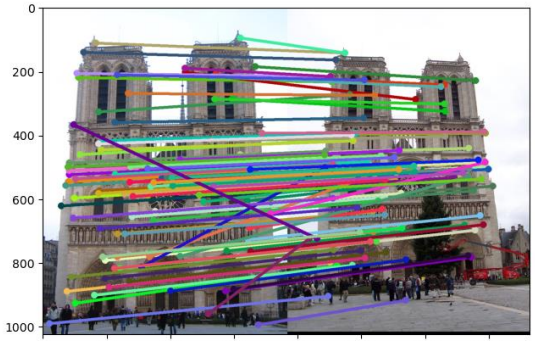
Why aren't normalized patches a very good descriptor?

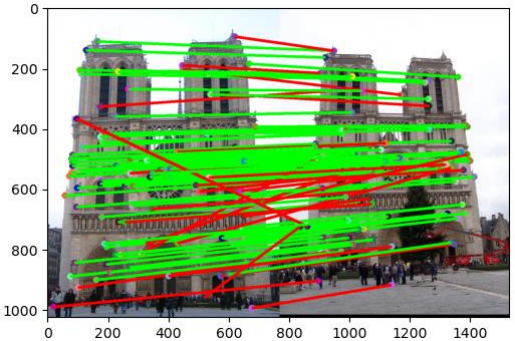
Many different locations can have similar raw pixel patches under lighting changes, so they aren’t robust to rotation/scale, minor misalignment, blur or repeating textures, causing ambiguous matches.

Part 2: Feature matching

Visualization of matches for Notre Dame image



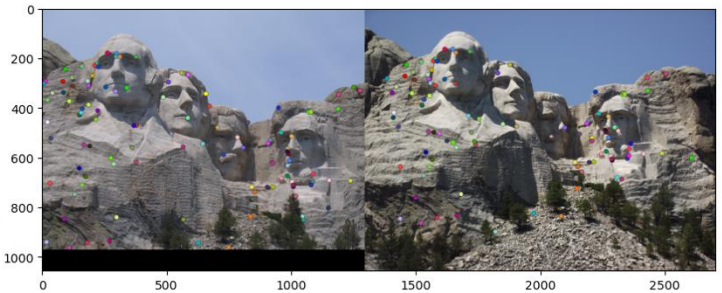


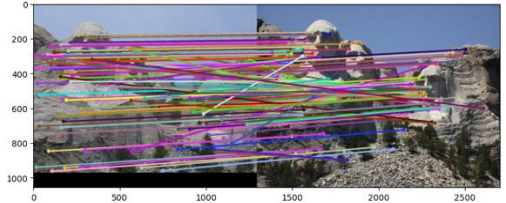


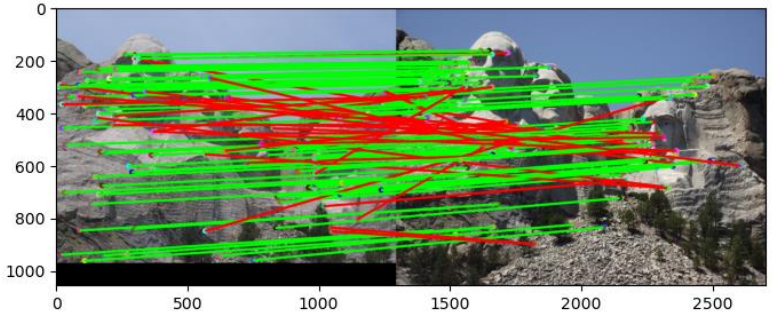
matches (out of 100): 107/100

Accuracy: 0.766355

Visualization of matches for Mt. Rushmore image pair





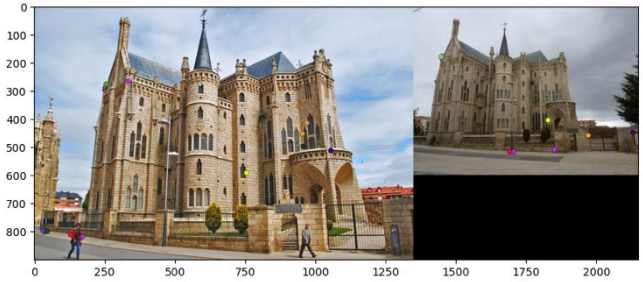


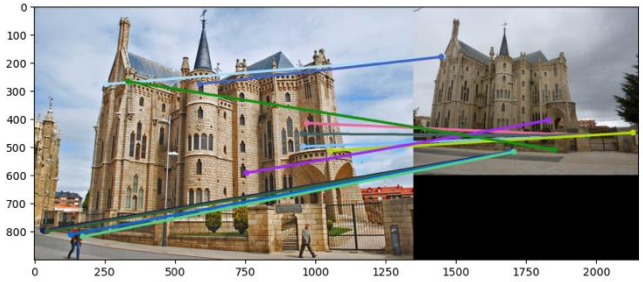
matches: 107/100

Accuracy: 0.728972

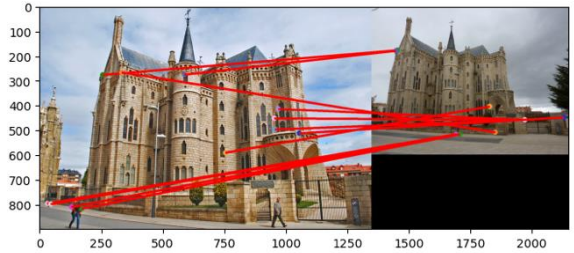
Part 2: Feature matching

Visualization of matches for Gaudi image pair





matches: 12/100
Accuracy: 0.000



How does your implementation of *match_features_ratio_test* filter and structure feature matches? What is the significance of the confidence score?

The ratio test matches by comparing each feature's best match distance to its second best match. A match is accepted if the closest neighbor is closer than the next closest one. This removes ambiguous matches that occur in repetitive textures, meaning the remaining matches are more reliable. The confidence score shows how distinctive a match is between the best and second best distances. In the results, images with clearer structural features like Notre Dame had more consistent matches unlike the Gaudi with fewer matches and lower accuracy.

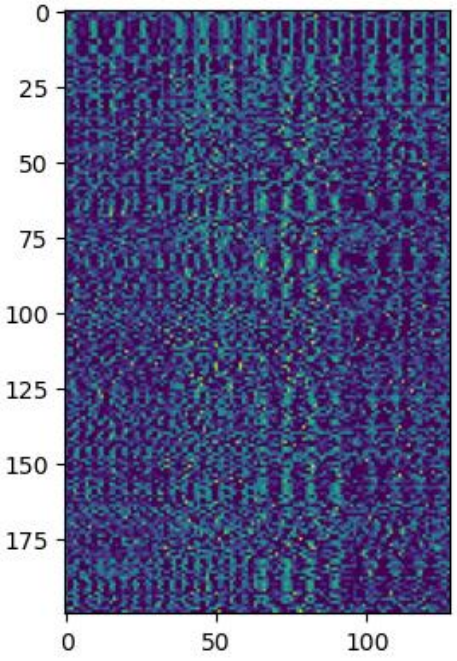
Part 2: Feature matching

Why do feature matches that easily pass the nearest neighbor distance ratio (NNDR) test tend to be more accurate?

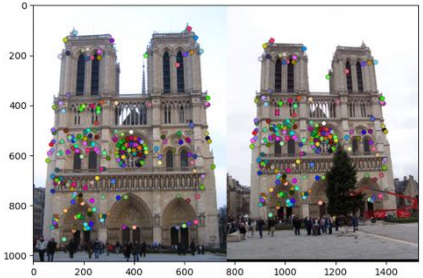
If the nearest neighbor is closer than the second nearest, the descriptor is more distinctive and less likely to come from repeating patterns or noises, so its more likely to be accurate.

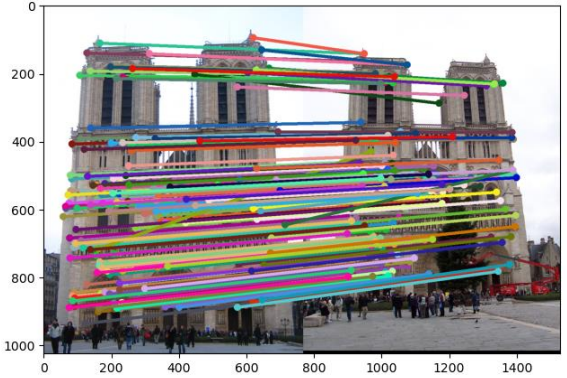
Part 3: SIFT feature descriptor

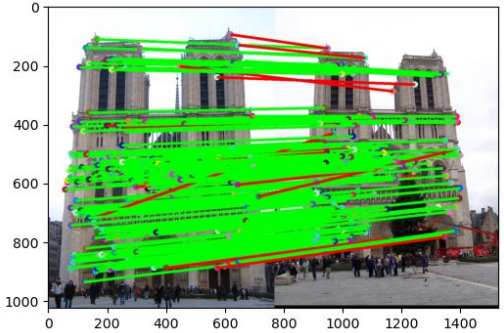
insert visualization of SIFT feature descriptor from proj2.ipynb here



insert visualization of matches for Notre Dame image pair





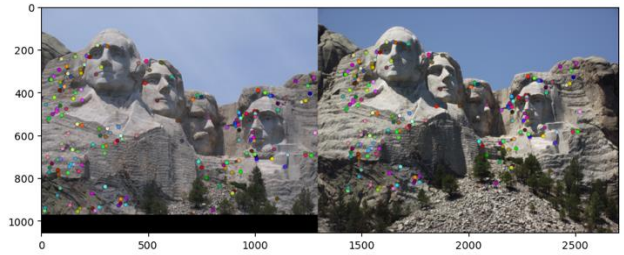


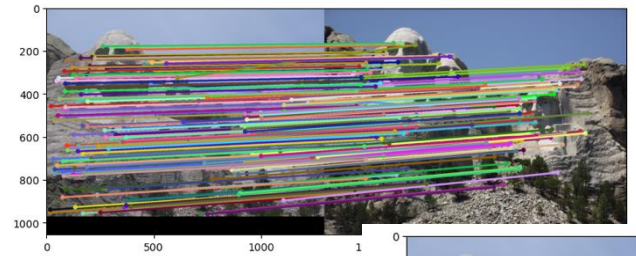
matches (out of 100): 197/100

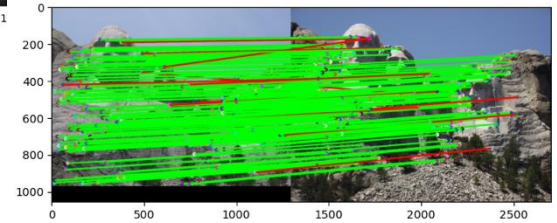
Accuracy: 0.918782

Part 3: SIFT feature descriptor

Visualization of matches for Mt Rushmore image pair



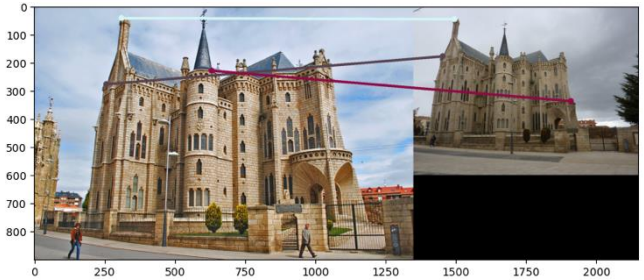


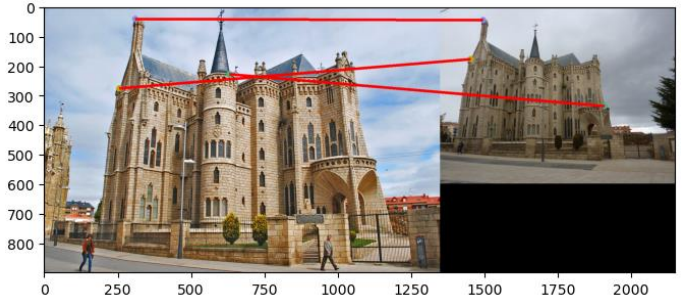


matches: 178/100

Accuracy: 0.932584

Visualization of matches for Gaudi image pair





matches: 3/100

Accuracy: 0.000

Part 3: SIFT feature descriptor

Explain how gradient histograms are constructed in the SIFT descriptor. What role do they play in the overall process?
You take a 16x16 patch around the keypoint, split it into a 4x4 grid of cells. In each cell, you build a 8 bin histogram of gradient orientations. Concatenating all 16 histograms yields a 128D vector that captures local structure and being robust to small shifts/noises.

Why are SIFT features better descriptors than the normalized patches?
They're based on gradients which are more stable than raw intensity. They also use local pooling which is robust to small misalignment. They also encode orientation distributions rather than exact pixels so they're better across illimitation changes.

What is the advantage of Square-Root SIFT?(1.5)
After the normalization, applying the square root reduces the dominance of large bins and improves robustness to repeated gradients and improves matching performance.

Part 3: SIFT feature descriptor

Why does our SIFT implementation perform worse on the given Gaudi image pair than the Notre Dame image and Mt. Rushmore pairs?

Gaudi has strong repetitive patterns and similar descriptors. The ratio test can fail or choose the wrong match because multiple candidate locations look almost identical, so the correspondences are ambiguous.

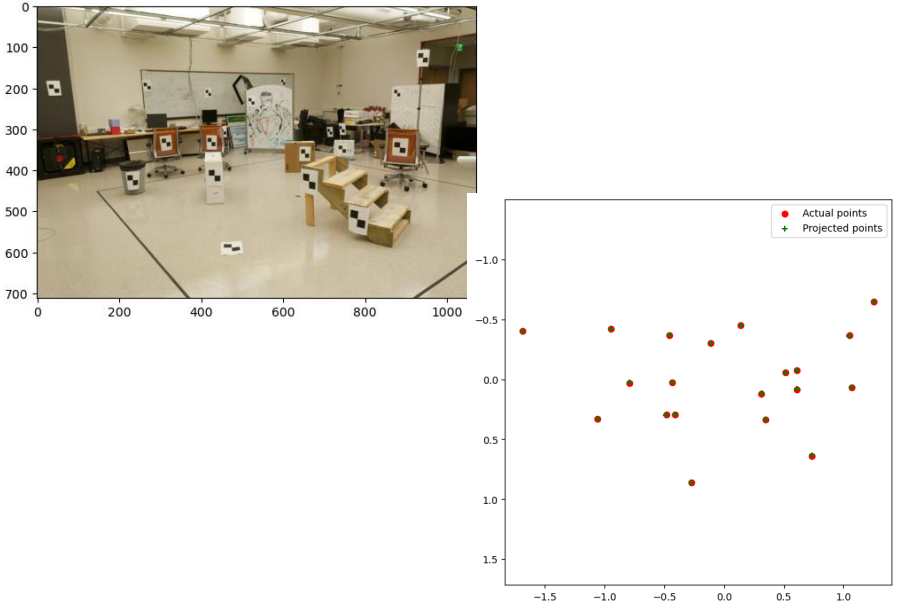
Part 3: SIFT feature descriptor

Why aren't our version of SIFT features rotation- or scale-invariant? What would you have to do to make them so?

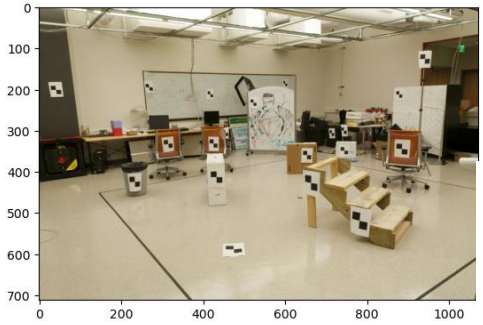
We use a 16x16 fixed window at a fixed scale and don’t rotate the descriptor to a dominant orientation. To make it scale invariant, detect the key points across scales and choose the best scale per key point. To make it rotation invariant, assign a dominant orientation per key point and rotate the sampling grid/histogram reference frame.

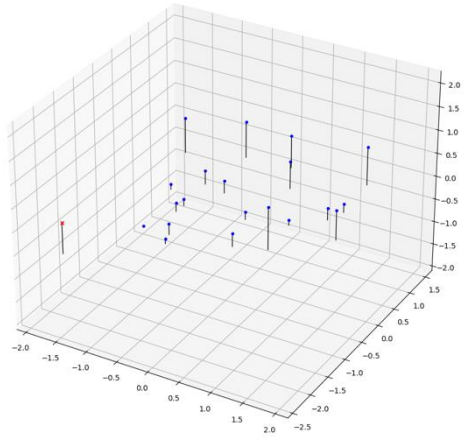
Part 4 (Extra Credit): : Projection matrix

insert visualization of projected 3D points and actual 2D points for the first CCB image we provided here



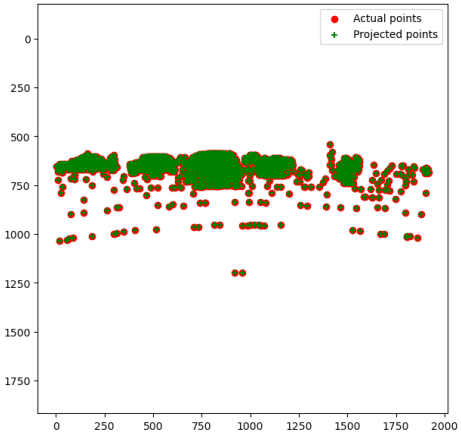
insert visualization of camera center for the first CCB image here



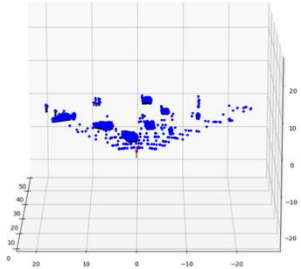


Part 4 (Extra Credit): : Projection matrix

insert visualization of projected 3D points and actual 2D points for the first Argoverse image we provided here



insert visualization of camera center for the first Argoverse image here



Part 4 (Extra Credit): : Projection matrix

What two quantities does the camera matrix relate?

It maps 3d world points (homogenous) to 2d image points (homogenous).

What quantities can the camera matrix be decomposed into?

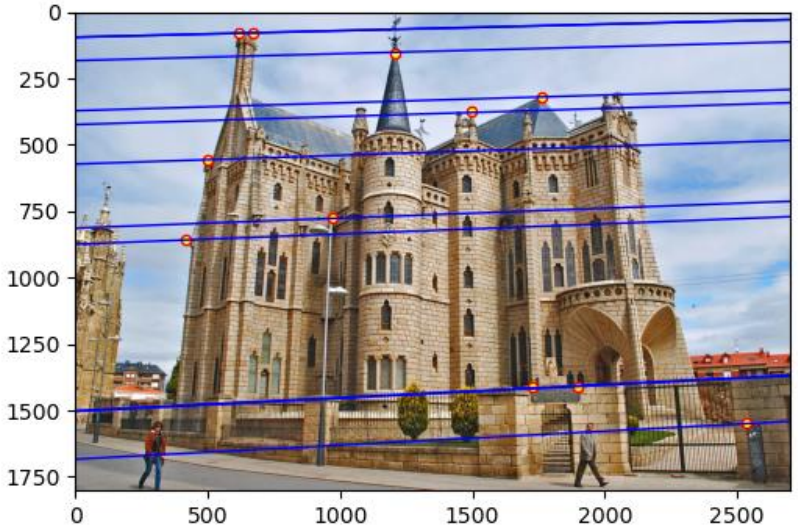
The intrinsic matrix K and extrinsic [R|t]

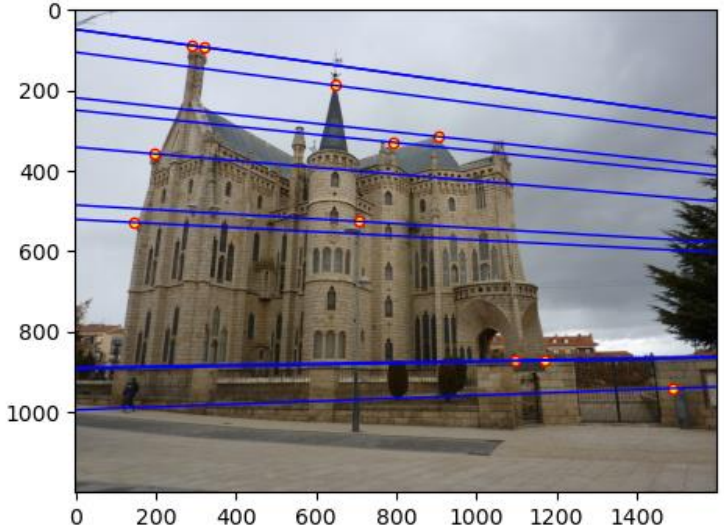
List any 3 factors that affect the camera projection matrix.

Focal length, principal point, camera rotation, camera translation, pixel skew

Part 5 (Extra Credit): Fundamental matrix

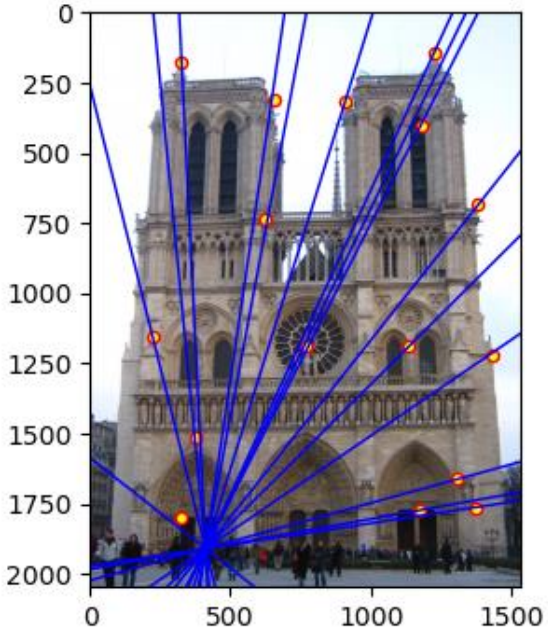
insert visualization of epipolar lines on the Gaudi image pair

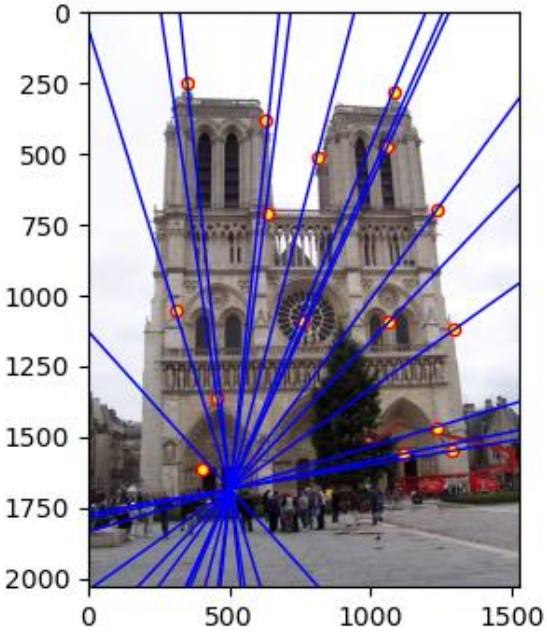




Part 5 (Extra Credit): Fundamental matrix

insert visualization of epipolar lines on the Notre Dame image pair





Part 5 (Extra Credit): Fundamental matrix

Why is it that points in one image are projected by the fundamental matrix onto epipolar lines in the other image?

A 3d point defines a ray through each camera center. The two camera centers and the 3D point defines the epipolar plane. That plane intersects the other image plane as an epipolar line and the corresponding point has to lie in it.

What happens to the epipoles and epipolar lines when you take two images where the camera centers are within the images? Why?

The epipoles can appear in an image, the epipolar lines radiate outside the epipole because all epipolar lines pass through the epipole by projecting through the other camera center.

Part 5 (Extra Credit): Fundamental matrix

What does it mean when your epipolar lines are all horizontal across the two images?

The pair is rectified, where the corresponding points lie on the same scanlines, simplifying stereo matching.

Why is the fundamental matrix defined up to a scale?

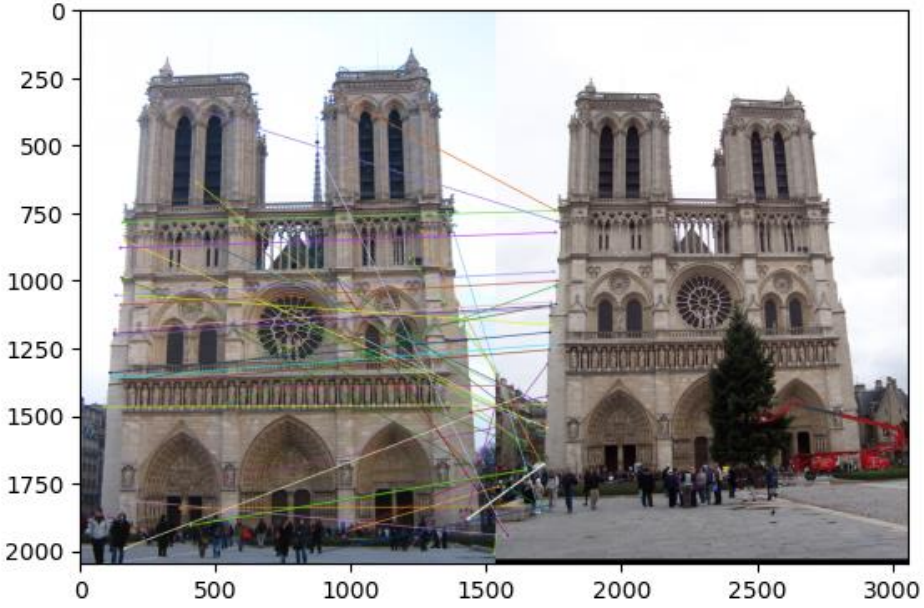
The epipolar constraint x^T is homogenous, so multiplying F by any nonzero scalar gives the same constraint.

Why is the fundamental matrix rank 2?

It must have a non trivial null space corresponding to the epipole so it cannot be a full rank.

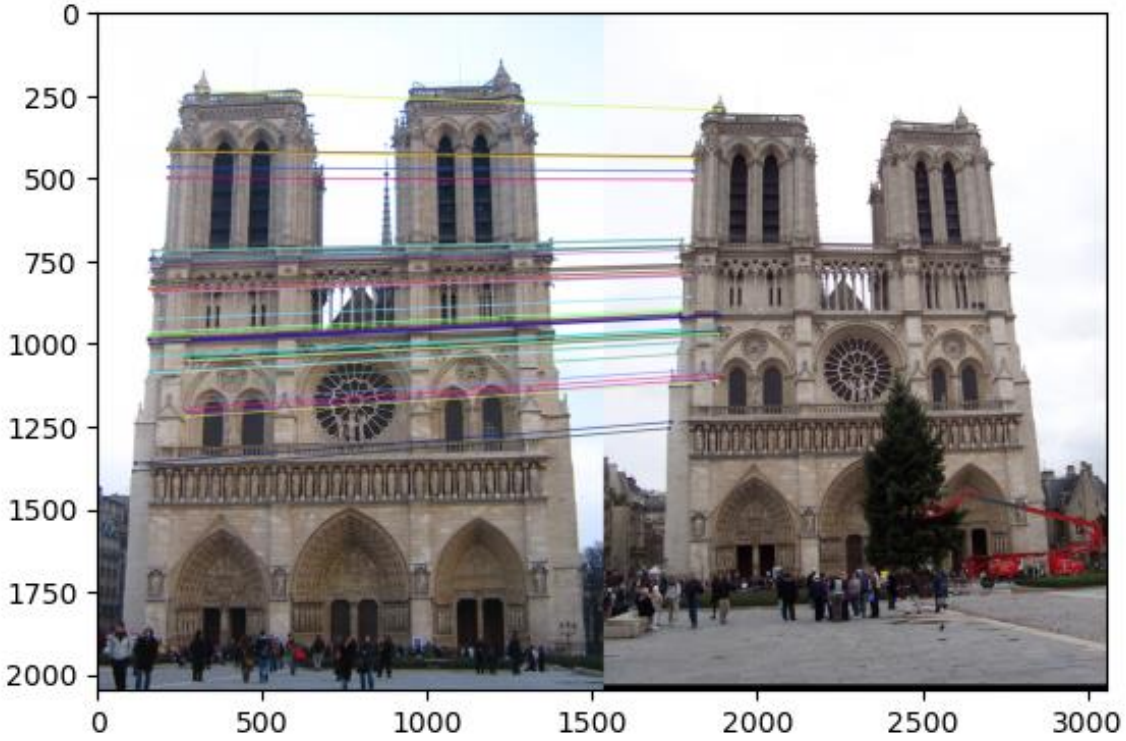
Part 6: RANSAC

insert visualization of correspondences on Notre Dame without RANSAC



Part 6: RANSAC

insert visualization of correspondences on Notre Dame with RANSAC



Part 6: RANSAC

How many RANSAC iterations would we need to find the homography with 99.9% certainty for your Notre Dame SIFT results? Assume a hypothetical inlier ratio (correspondence accuracy) of 90% and that you are using the minimal sample size of 4 points

1- 0.9^8 = 0.3439. N = log(0.001)/log(0.3439) = 6.47

7 iterations

One might imagine that if we have many correct matches, it would be better to use more than the minimum number of points to compute the homography in each sample. Investigate this by finding the number of RANSAC iterations you would need if you used a sample size of 8 points instead of 4 (keeping the same 99.9% certainty and 90% inlier ratio.

1- 0.9^8 = 0.5695. N = log(0.001)/log(0.5695) = 12.27. 13 iterations

If our dataset had a lower point correspondence accuracy (i.e., a lower inlier ratio), say 70%, what is the minimum number of RANSAC iterations needed to find the homography with 99.9% certainty? Remember to use the minimal sample size of 4 points in your calculation.

0.7^4 = 0.2401

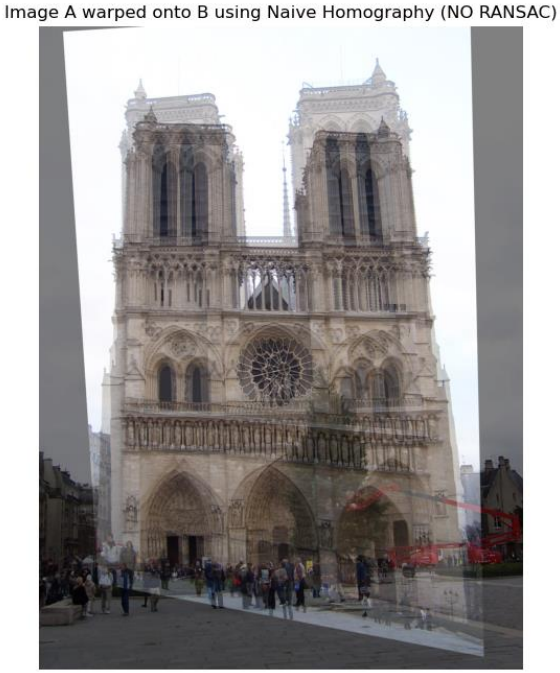
1 – w^4 = 0.7599

N = log(0.001)/log(.7599) = 25.1

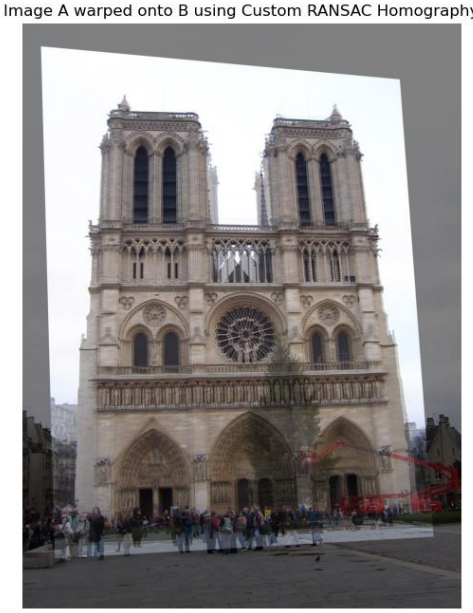
26 iterations

Part 6: Performance comparison

insert visualization of the warped and blended image pair without RANSAC



insert visualization of the warped and blended image pair using RANSAC



Part 6: Performance comparison

Describe the different performance of the two methods.
Without RANSAC, the homography is estimated using all matches, including incorrect ones. This produces warped images with distortions. With RANSAC, incorrect matches are removed before computing the transformation, resulting in a much cleaner and accurate alignment.

Why do these differences appear?

Naive estimation assumes all correspondences are accurate. Outliers introduce large errors that skew the least squares solutions. RANSAC repeatedly samples subsets and selects the transformation supported by the most consistent correspondences and ignoring outliers.

Which one should be more robust in real applications? Why?

RANSAC is more robust because it models the possibility of incorrect correspondences and isolates inliers before estimating the final transformation. This makes it reliable even when several matches are wrong.

Conclusion

What made the RANSAC-based homography estimation significantly more effective than the naive homography estimation? In your answer, discuss the role of outliers in the initial SIFT matches and explain how the two methods differ in handling them.

The initial SIFT matches include accurate correspondences and incorrect ones caused by repetitive textures or noises. A naive homography estimation uses all the matches, so even a small number of outliers can distort the transformation. RANSAC improves performance by repeatedly sampling subsets of matches, estimating candidate homographies, and selecting the one supported by the largest number of inliers. Because it ignores outliers during model fitting, the final homography is accurate, which leads to better results.